

Query Processing

Selection Operation

1 Introduction to File Scans

In query processing, the **file scan** is the lowest-level operator used to access data. File scans are search algorithms that locate and retrieve records satisfying a given selection condition. In relational systems, a file scan allows an entire relation to be read when it is stored in a single dedicated file.

Key Terminology

- b_r : Number of blocks containing records from relation r
- h_i : Height of the B⁺-tree index
- t_S : Time for one disk seek (random access latency)
- t_T : Time to transfer one block
- n : Number of matching records

2 Selection Algorithms

The system can perform selection in several ways, depending on available indices and the type of condition. Below are the four fundamental strategies.

2.1 A1. Linear Search (File Scan)

Algorithm: Scan each file block and test all records to check whether they satisfy the selection condition.

Cost Estimate:

$$\text{Cost} = b_r \times t_T + 1 \times t_S$$

Special Case (Key Attribute): Scanning can stop once the matching record is found:

$$\text{Cost} = \frac{b_r}{2} \times t_T + 1 \times t_S$$

Characteristics:

- Can be applied regardless of selection condition or ordering.
- Works even without any index.
- Binary search is generally **not suitable** because data is often not stored consecutively and would require multiple seeks.

Example:

```
SELECT * FROM instructor WHERE salary < 70000;
```

System scans all blocks of the `instructor` table sequentially.

Remarks: Slowest method, but universally applicable.

2.2 A2. Clustering Index (Equality on Key)

Algorithm: Retrieve a single record that satisfies the equality condition on a key attribute using a clustering (primary) index.

Cost:

$$\text{Cost} = (h_i + 1) \times (t_S + t_T)$$

Formula Breakdown:

1. $h_i \times (t_S + t_T)$: Traverse h_i index levels (each requires one seek + one transfer).
2. $1 \times (t_S + t_T)$: Fetch the actual data block containing the record.

Example:

```
SELECT * FROM student WHERE student_id = 'S123';
```

B⁺-tree index on `student_id` is traversed directly to find the record.

Remarks: Highly efficient for unique key lookups; index height is typically small ($h_i \approx 1$ if cached).

2.3 A3. Clustering Index (Equality on Non-Key)

Algorithm: Retrieve multiple records using a clustering index when the condition is an equality on a non-key attribute. Records are stored consecutively because the file is sorted on the clustering key.

Cost:

$$\text{Cost} = h_i \times (t_S + t_T) + t_S + b \times t_T$$

Formula Breakdown:

1. $h_i \times (t_S + t_T)$: Traverse the B⁺-tree index to the first matching leaf entry.
2. $1 \times t_S$: One seek to position the disk head at the start of the contiguous block range.
3. $b \times t_T$: Sequentially read the b consecutive data blocks containing all matching records.

Example:

```
SELECT * FROM instructor WHERE dept_name = 'CSE';
```

All CSE instructor records are on consecutive blocks.

Remarks: More efficient than linear scan since matching records are contiguous.

2.4 A4. Secondary Index (Equality)

Algorithm: Use a secondary (non-clustering) index to find records satisfying equality conditions on the search-key.

Case 1 – Equality on Key:

$$\text{Cost} = (h_i + 1) \times (t_S + t_T)$$

Case 2 – Equality on Non-Key:

$$\text{Cost} = (h_i + n) \times (t_S + t_T)$$

Each of the n matching records may be on a different block.

Formula Breakdown (Non-Key):

1. $h_i \times (t_S + t_T)$: Find the leaf entries in the secondary index.
2. $n \times (t_S + t_T)$: Each of the n records requires a separate random I/O because data is not physically ordered by this key.

Example:

```
SELECT * FROM instructor WHERE city = 'Dhaka';
```

Secondary index on `city` locates all matching records, possibly scattered across blocks.

Remarks: Can become expensive if many records match; often mitigated by buffer caching.

3 Concept: Access Paths

Index structures are called **access paths**, as they define how data can be located efficiently.

Index Type	Description
Clustering Index	Records are stored in physical order of the index key. Only one clustering index can exist per relation.
Secondary Index	Records are not stored in index key order. The index contains pointers to scattered data blocks. Multiple secondary indices can exist.

4 Summary and Comparison

Selection Methods Overview

Method	Type	I/O Pattern	When Applicable
A1	Linear Search	Sequential	No index available
A2	Clustering (Key)	Random	Equality on key
A3	Clustering (Non-Key)	Mixed	Equality on non-key
A4	Secondary Index	Random (n)	Equality on any indexed field

5 Reference: Typical Access Costs

For 4 KB Blocks:

- **High-end Magnetic Disk:** $t_S = 4 \text{ ms}$, $t_T = 0.1 \text{ ms}$
- **SSD:** $t_S = 20\text{--}90 \mu\text{s}$, $t_T = 2\text{--}10 \mu\text{s}$

Note: In practice, index internal nodes are cached, so effective $h_i \approx 1$.